

Genetic Algorithm for Large Dynamic Vehicle Routing Problem on GPU

A. Benaini¹, A. Berrajaa^{1,2}

¹ Normandie Universite, Lab. LMAH, 25 Rue Philippe Lebon, 76063 le Havre, France

² Universite Mohammed Premier, Lab. LaRI, Oujda Maroc

abdelhamid.benaini@univ-lehavre.fr, berrajaa.achraf@gmail.com

Abstract—Vehicle Routing Problems (VRPs) are fundamental optimization problems of distribution logistics and transportation systems. In the real-world, VRPs are dynamic in the sense that new customer requests continuously arrive over time, after a number of vehicles have already started their tours. These new requests must be incorporated into already planned routes. Dynamic VRPs require making decisions as fast as possible. This needs solution methods with high computational efficiency especially for large size problems. The aim of this paper is to attempt to achieve this objective. So, we design a new genetic algorithm for the Dynamic VRP and we implement it on Graphics Processing Units (GPU). The proposed approach is based on a simple method that inserts new requests in already planned routes then it optimizes the resulting solution via genetic operators that we introduced. We evaluated its effectiveness on some published benchmarks and on our own large instances (up to 3000 nodes) and we compared it to other recent works on Dynamic VRPs.

Index Terms—Dynamic Vehicle Routing Problem, Insertion heuristic, genetic algorithm, GPU.

I. INTRODUCTION

The static VRP deals with a deterministic operational environment where all information is known before optimization (at planning time) and remains static during the execution of the routing plan. It may be briefly described as follows. Given, one or more depots, a fleet of vehicles, homogeneous or not, and a set of customers with known demands, find a set of closed routes, originating and ending at one of the depots, to service all customers at minimum cost, while satisfying vehicles capacity and eventually time windows constraints. In Dynamic Vehicle Routing Problems (DVRP), also referred to as online or real-time VRP, new requests are received as time progresses and must be dynamically incorporated into already planned routes while vehicles have already started their tours. Vehicles do not have to go back to the depot in order to treat new requests assigned to them when they have already left the depot. Ideally, the new requests should be inserted quickly and efficiently into the already planned routes without re-planning the non visited parts of the routes. Here, it is assumed that there is a continuous communication system between the planner and vehicles in order to monitor execution of routes and implement real-time control strategies, i.e. fast re-optimization of the routing plan.

DVRP occurs in many routing problems in the real world: services, maintenance operations, transport of goods or people, telecommunications, production planning, and so on. See [1], [2], [3], [4], [5], [6] for a comprehensive review of the DVRP and [7] for a survey on results achieved on the different types of DVRP.

DVRP requires making decisions as fast as possible (in real-time) because the time available to reach a decision is limited. So this needs solution methods with high computational efficiency especially for problems with large number of customers. Since the DVRP is NP-hard problem, meta-heuristics like the Genetic Algorithm, Ant Colony, Local Search, ... are used to solve such problems rather than exact methods. Even these meta-heuristics remain insufficient for finding "good" solution quickly for large DVRPs. Nevertheless, the use of effective resolution method and modern Graphics Processing Units (GPUs) can achieve this goal.

GPUs are actually available in most of personal computers and are greatly efficient that traditional CPUs in terms of computational time. They are multiprocessor devices, offering hundreds of computing cores working as SIMD massively parallel machines. However, few GPU implementations for solving optimization problems have been proposed. Campeotto et al [8] present an interesting GPU implementation of large neighborhood search for solving constraint optimization problems. Arbelaez et al. [9] propose a GPU implementation of parallel constraint-based local search. Uthayopas et al. [10] and Benaini et al [11] solve the VRP with/without time windows. Talbi et al. [12], [13] established a guideline to design and implement efficiently local search metaheuristics on GPU. They propose an efficient mapping of the iteration-level parallel model on the hierarchical GPU. In [12] the authors study the GPU computing for parallel local search metaheuristic algorithms and genetic algorithms for solving VRP. Diego et al. [14] propose a GPU strategy to solve the VRP with ant colony optimization heuristic.

Two main approaches are used to solve the DVRP as stated in [7].

i) *The periodic reoptimization* approach gives a first optimal initial routing plan (set of routes) before the beginning of the

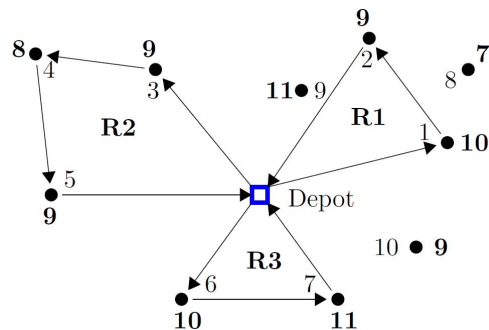
workday. After the transportation has begun, an optimization procedure periodically solves a static problem corresponding to the current state, either whenever the available data change. Generally, the planning horizon is divided into periods and the DVRP is solved by solving a static VRP at each period. [15] uses this approach and solves each single VRP by a Genetic Algorithm.

This paper presents a Genetic Algorithm for solving large DVRP with periodic reoptimization approach. The proposed approach is based on a simple method that inserts new requests in already planned routes then it optimizes the resulting solution via genetic operators that we introduced. We evaluated its effectiveness on some published benchmarks and on our own large instances (up to 3000 nodes) and we compared it to other recent works on Dynamic VRPs among them the Improved Variable Neighborhood Search (IVNS) algorithm [5], the Genetic Algorithm (GA-DVRP) [15], the Ant Colony System (DVRP-ACS) [2] and the Iterated Local Search based metaheuristic algorithm (ILS-SP) [16].

II. PROBLEM STATEMENT

This strategy has been used in several works on DVRP. Xu [5] proposed an Improved Variable Neighborhood Search

Let $s = R_1, \dots, R_m$ the solution (routes) of the static VRP (before the start of the working day), m the number of routes, n the number of customers $1, \dots, n$ with the depot 0. Let $R_i = (0, v, \dots, v_k, 0)$. Then $demand(R_i) = demand(v_1) + \dots + demand(v_k)$ and $cost(R_i) = d(0, v_1) + d(v_1, v_2) + \dots + d(v_k, 0)$; where $d(i, j)$ is the distance between nodes i and j supposed to satisfy the triangular inequality. The cost of the solution s is $\sum_{i=1}^m cost(R_i)$.



We propose to solve the static VRP at each period in two steps : (1) insert the new requests in the already planned routes at the previous period, eventually create new routes (2) improve this solution using a GA with initial population generated from this solution.

III. INITIAL SOLUTION

Each customer L_i is inserted in its closest route R_j if the vehicle's capacity allows it i.e if $demand(R_j) + demand(L_i) \leq the\ capacity\ of\ the\ vehicle\ serving\ R_j$. Else try to insert it in the second closest route to it etc. On the other hand, L_i is inserted between two consecutive nodes u^* and v^* of the route which minimize $d(u, L_i) + d(L_i, v) - d(u, v)$ for all u, v two consecutive nodes of the route. If L_i cannot be inserted in any route then a new route will be created for it.

The Figure 2 shows the initial solution R_1, R_2, R_3 at period 1 of the problem represented in Fig. 1. We obtain this solution by inserting the 3 new costumers 8, 9, 10 in the solution R_1, R_2, R_3 as follows. Assume that the sorted list is $L = 8, 10, 9$. Then first insert the customer 8 in R_1 assuming that R_1 is the closest route to node 8 and the capacity of the vehicle serving R_1 allows this insertion. Next, insert the customer 10 in R_3 assuming that R_3 is the nearest route to 10 and the capacity of the vehicle serving R_3 allows this insertion. Finally, insert the customer 9 in R_1 assuming that R_1 is the closest route to node 9 and the capacity of the vehicle serving R_1 allows this insertion. Note that the period 1 starts with vehicle 1 positioned at node 1, vehicle 2 positioned at node 3 and vehicle 3 positioned at node 6 and that this initial solution will be improved by the GA.

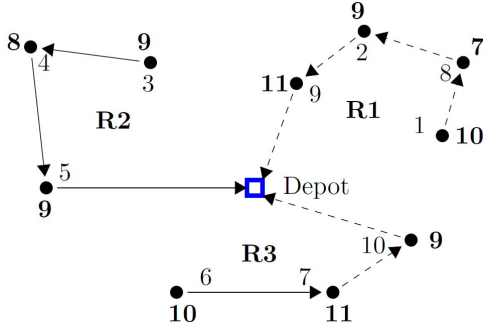


Fig. 2. Initial solution of the DVRP of figure 1

IV. THE GENETIC ALGORITHM

Genetic Algorithm is a kind of evolutionary methods which simulates natural process of biological evolution. In GA a new population is produced from the previous population by using genetic operators iteratively (generally, with stop after a given number of iterations). The execution time of the GA can become a limiting factor for large size problems, because a lot of candidate solutions must be evaluated. Since GAs are well suited for computing on GPUs, we opted for GA as a heuristic improvement. Nevertheless, we use genetic operators similar to those used in other heuristics (k-opt, relocate, etc). An important aspect of GA is the choice of solution representation or chromosome encoding.

Here, we use the Set-Based-Representation introduced in [17] (for the hub location problem) to encode a solution of the DVRP as follows. Each route R_i of the solution is represented

by the ordered set of nodes (cluster) C_i that make up the route and the solution is encoded by the clusters $C_1, \dots, C_m$. The first number in each cluster is used to identify the position of the vehicle. This encoding gives explicitly the routes solution. So, it does not need a decoding step.

For instance, the encoding of the solution of Figure 2. is $C_1 = (1, 8, 2, 9, 0)$, $C_2 = (3, 4, 5, 0)$, $C_3 = (6, 7, 10, 0)$ meaning that the initial positions of vehicles are 1, 3, 6 and that the customers 1, 3, 6 are already serviced.

A. Genetic operators

We define simple genetic operators that are iteratively applied to the encoding of an initial solution in order to improve it.

1) *Crossover*: The crossover is the main operator of a GA and the most complicated to define. The following four steps define the crossing of two chromosomes p and p' .

step 1. Select randomly a cluster C from p and C' from p' .

step 2. Make a random cut in two points of the selected clusters C and C' . Let X (resp. X') be the set of nodes between the two cut points of C (resp. of C'). Basically, X (resp. X') is the cut in C (resp. in C'). The length (number of nodes) of the cuts X and X' must be the same. The first node of C and the first node of C' must not be part of the cuts. For the sake of simplicity, we limit the length of the cut to 1 or 2 or 3. This will allow to swap either a single node or an arc of 2 nodes or a path of 3 nodes between C and C' .

step 3. Replace X with X' in C and X' with X in C' if this does not violate the capacity constraint producing two solutions (children) ch and ch' . Otherwise, goto step 1.

step 4. The nodes in $Y = X \setminus X \cap X'$ appear twice in ch' and the nodes in $Y' = X' \setminus X \cap X'$ appear twice in ch . This must be corrected as follows. For each $i \in Y$, find $j \in Y'$ and replace i by j in ch' if this does not violate the capacity constraint and set $Y' = Y' \setminus \{j\}$. Replace the duplicates of ch in the same way. If these replacements prove impossible because of the capacity constraint then goto step 1.

An example of the crossover is shown in Figure 3. C_2 is selected from p and C_1 from p' . The length of the cut is 2. The cut in p is $X = \{4, 5\}$ and the cut in p' is $X' = \{1, 4\}$. So, $Y = \{5\}$ and $Y' = \{1\}$ means that node 5 (resp. node 1) appears twice in p' (resp. in p) and must be replaced by 1 (resp. by 5).

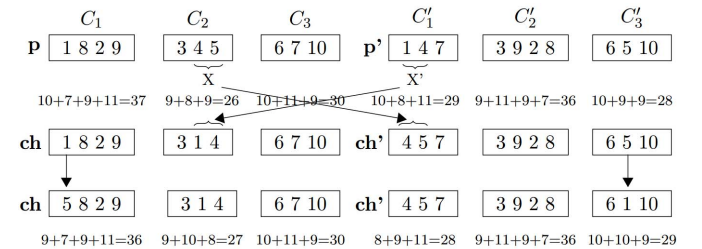


Fig. 3. An example of crossover

2) *Mutation*: The objective of the mutation operator is to enlarge the search space to avoid local minima. The mutation is applied to 2% of the current population and consists in moving a node from one cluster to another cluster of the same solution (the target clusters are those with minimum number of nodes). Modified clusters must be corrected as for crossover operator.

3) *Re-schedule*: The cross and the mutation operators move some nodes to some clusters (routes); The aim of the re-schedule operator is to re-insert these nodes in "best positions" of these routes. Let k be a node moved to C . Then find u^* and v^* in C that minimize $d(u, k) + d(k, v) - d(u, v)$ for all u and v in C and insert k between u^* and v^* . Do this for all clusters where some nodes are moved.

Note that the served customers at the previous period are removed from the chromosome representation and the new requests of the current period are added to the chromosome representation. Therefore, the size of chromosomes generally remains the same over the periods.

V. GPU IMPLEMENTATION

Now we outline the GPU/CUDA principles and the GPU implementation of the proposed algorithm.

A. Basic concepts in GPU computing

Graphics Processing Units GPU are actually available in most of personal computers. They are used to speed up the execution of variety of problems such as image processing, scientific computing, combinatorial optimization. A GPU consists of a set of Streaming Multiprocessors SMs each consists of a large number of Streaming Processors SPs. For instance, 2 SMs and 192 SPs per SM for our Quadro, 15 SMs and 192 SPs per SM for the Tesla. The smallest unit in the GPU that can be executed is called thread. All threads execute the same code. The threads running within an SM can synchronize with each other and two threads located in different SMs cannot synchronize with each other. Threads are grouped into blocks of equally size and blocks are grouped into grid (blocks are independent and cannot be synchronized). The hardware will assign an entire block to a single SM, though several blocks can run in the same SM. The programmer controls the block size and the number of blocks, and writes the code to take advantage of how the hardware works. A simplified GPU architecture is given in Figure 4 where the rectangles of thread blocks are for illustration purposes and are not a physical part of the architecture. In this figure SM stands for Streaming Multiprocessor and SP for Streaming Processor (core).

Parallel execution and thread management is automatic. An SP can execute concurrently 32 threads of the same block (warp); a block consists of one or more warps. The number of thread blocks can greatly exceed the number of processors. CUDA virtualizes the processors and gives the programmer the flexibility to parallelize at whatever granularity is most convenient. Also, it allows the same program to scale to

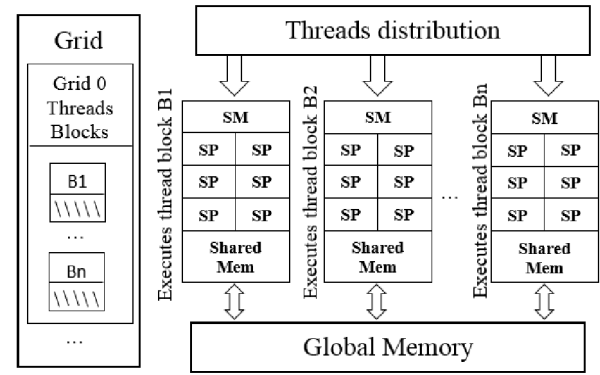


Fig. 4. Simplified GPU architecture

widely varying numbers of processor cores.

For instance, a grid of 6 blocks will be executed on two types of GPUs : GPU1 with 2 SMs and GPU2 with 3 SMs as follows. Each SM of the GPU1 executes sequentially 3 blocks : SM0 executes block0, then block2 and finally block4, SM1 executes block1, then block3 and finally block5. Each SM of the GPU2 executes sequentially 2 blocks : SM0 executes block0 then block3, SM1 executes block1 then block4 and SM2 executes block2 then block5. Each block is executed in parallel by the SPs of the corresponding SM.

In addition to the GPU/SM/SP and grid/block/thread/warp hierarchies, another key hierarchy is memory structure. The most important three types of memory are shared, global and constant.

- *Shared memory* : All the threads in an SM share this memory and use it to communicate among themselves. Access is very fast (2 cycles) as this memory is on-chip. On the other hand, shared memory is small, currently 48 Kb per SM, and the data stored in it are valid only for the life of the currently-executing kernel (block). If several blocks are assigned to the same SM then they are independently executed (but sequentially) by the SM and the shared memory is entirely allowed to the running block.

- *Global memory* : This is shared by all the threads in an entire application, and is persistent across kernel calls, throughout the life of the application. It is usually much larger than shared memory (1-4 Gb) and it is accessible from the host. On the other hand, global memory is off-chip and very slow, taking hundreds of clock cycles per access (400 cycles).

- *Constant memory* : it is used to store data constant. Constant memory is small (64 Kb for the quadro) and accesses to it is fast (about 20 cycles).

Accesses to global, shared and constant memory are done via half-warps. It is therefore important to avoid storing, in the global memory, data that must be accessed multiple times by some threads of a block. Its better to copy it to shared memory (eventually to constant memory). A typical CUDA program is a C program where the functions are distinguished based on whether they are meant for execution on the CPU or on the

GPU. A more detailed presentation of GPU and CUDA can be found in [18].

B. GPU implementation of the DVRP

Given a "good" solution $s = C_1, \dots, C_m$ of the static VRP (before the arrival of the nr new requests) with m routes and n nodes. Copy the distance matrix to the global memory of the GPU because neither shared memory nor constant memory can store this matrix for large VRP. Each new request is processed by a thread, so nr threads are required. As the number of threads per block is limited to 1024 we define these threads as a grid of $g = \lceil \frac{nr}{1024} \rceil$ blocks each of $t = \lceil \frac{nr}{g} \rceil$ threads. With the syntax of Cuda, this is defined by : $dim3 \ dimBlock(t, 1)$ and $dim3 \ dimGrid(g, 1)$. Next, copy the chromosome $(C_1, \dots, C_m)$, the n static demands and the nr dynamic demands to the shared memory of each block.

- First, generate a new feasible solution by inserting the new requests in the solution s starting by requests with small demands. This step is implemented on the previously defined block as follows. Thread i computes the nearest clusters to the i -th new customer and cooperates with the other threads to insert the i -th new customer in the best position of its nearest cluster (route). If a new request k cannot be inserted in any cluster (because of the capacity constraint) then create a new cluster $(0, k, 0)$. This constitute the initial solution p of the DVRP composed of at most $m + nr$ clusters.

- The solution p is improved by the GA as follow. Define R blocks $block_0, \dots, block_{R-1}$ each with M threads where M will be defined later. Copy p to the shared memory of each $block_i$. All blocks work in parallel each on a pair of parents. The $block_i$, $0 \leq i < R$, executes the six following tasks.

1) Select randomly two clusters C and C' from p and exchanges one node of C with another of C' resulting in a feasible solution p_i .

2) Repeat this process a second time to produce p'_i . A not feasible solution is rejected and two other clusters/nodes are randomly chosen.

3) Cross p_i, p'_i to produce two offspring ch_i, ch'_i .

4) Re-schedule ch_i, ch'_i and select the two individuals q_i, q'_i with the best fitness among p_i, p'_i, ch_i, ch'_i .

5) Apply the mutation operator (if any) to q_i and/or q'_i

6) Set $(p_i, p'_i) = (q_i, q'_i)$ for the next iteration if the stop criterion is not verified. Otherwise, the program returns the individual with the best fitness among all the p_i, p'_i .

Note that, at each iteration, each couple of parents (p_i, p'_i) is treated, so the only selection is the formation of couples (p_i, p'_i) .

To compute quickly the fitness, we store with the encoding of individual the cost of the corresponding clusters in the shared memory of the block. The block only re-computes the cost of the modified clusters during the iteration by accessing the distance matrix stored in global memory.

VI. EXPERIMENTAL TESTS

We implemented the previously presented approach on GPU Nvidia Quadro with 2 GB and 2 SM and 384 cores running under CUDA 7.5 environment. The maximum number of blocks used is $R = 288$. This allows to process a population of 576 individuals in parallel. The number of iterations (stop criterion of GA) varies according to the size of the instances and is limited to is 200 for large instances.

In order to assess the performance of our GPU implementation, three test problems with respect to different sizes (small, medium, and large) are done. A comparison of our results with those of Xu [5] and Montemanni et al. [2] is provided. We used the benchmarks proposed by these authors with customers ranging from 50 to 150, the large realistic benchmarks with sizes from 100 to 1000 proposed by Uchoa et al. [16] and our own instances with up to 3000 customers. The results reported in Tables 7-11 are taken over 10 runs of the GPU implementation.

A. Small benchmarks

Xu [5] proposed an example where the distribution area is a square of $50 \text{ km} \times 50 \text{ km}$; 30 static demand customers and 10 new requests randomly generated. Each customers demand is a random number of $[0, 2]$, the vehicles capacity is equal 8t, the location of the depot is (25 km, 25 km). The coordinates and demands of static customers and new customers are shown in Tables 1 and 2.

TABLE I
THE POSITION AND DEMAND OF STATIC CUSTOMERS

No.	1	2	3	4	5	6	7	8	9	10
X	29	47	45	32	6	34	44	36	38	5
Y	44	9	13	16	1	6	6	46	39	28
D	0.5	0.2	0.4	2	1.7	0.3	1.7	0.7	0.2	1.2
No.	11	12	13	14	15	16	17	18	19	20
X	27	32	17	40	37	26	34	18	43	19
Y	7	3	14	47	47	12	19	40	26	3
D	1.7	0.5	0.3	1.5	0.4	0.8	1.4	1.4	1	1.7
No.	21	22	23	24	25	26	27	28	29	30
X	25	26	29	29	14	50	47	8	42	46
Y	49	49	3	37	37	10	26	10	27	34
D	0.2	0.9	0.3	0.2	0.5	0.3	0.9	0.5	1.4	2

TABLE II
THE POSITION AND DEMAND OF THE NEW CUSTOMERS

No.	A	B	C	D	E	F	G	H	I	J
X	8	47	2	9	21	41	39	4	44	13
Y	19	10	15	9	12	43	25	8	41	49
Period	1	2	2	1	1	1	1	2	1	2
D	1	1.5	1.6	3	1.4	0.3	0.2	2.2	2.8	1

In period 1, six new requests (A, D, E, F, I, G) arrive and will be incorporated in the solution obtained before the start of the working day. At period 2, the four new requests (B, C, H, J) will be incorporated in the solution of period 1. Tables 3 – 6 report the results obtained by our approach and those proposed in [5]. Note that in the results of Xu [5] there are routes that exceed the capacity of the vehicle fixed to 8t (see

Tables 3 and 5). The columns *utilization%* in these tables give the used percentage of the vehicle capacity. A percentage $> 100\%$ means that the demand of the route exceed the vehicle capacity and therefore the non feasibility of the solution.

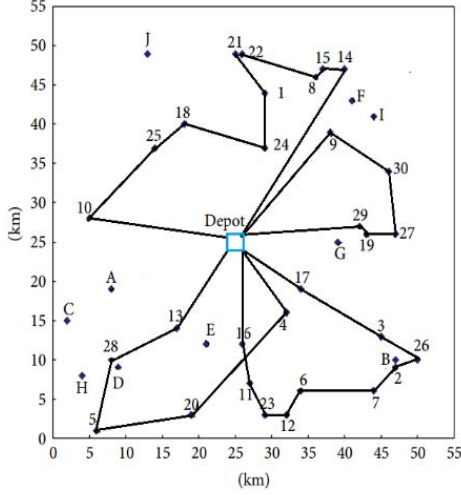


Fig. 5. Initial solution

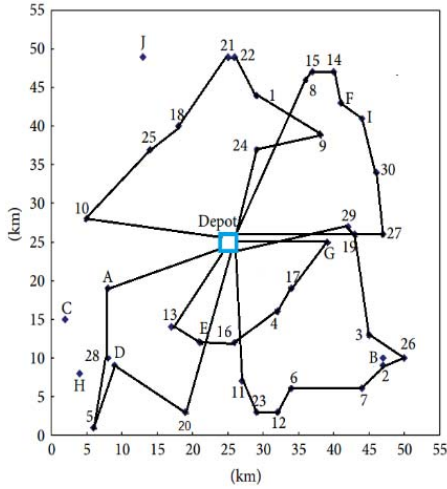


Fig. 6. Solution for the period 1

Figures 5 – 7 shows the specific position of the customers and the routes planning for the three steps : the initial distribution, the solution for the period 1 and the solution for the period 2 obtained by the GPU implementation.

TABLE III
THE IVNS SOLUTION [5] FOR THE PERIOD 1.

Vehicle	Route	Utilization %	Cost/Km
1	0-17-7-3-2-26-27-19-29-G-0	93.75	81.22
2	0-4-6-12-23-11-16-0	70	50.81
3	0-24-1-22-21-18-25-10-0	61.25	75.83
4	0-A-D-28-5-20-E-13-0	> 100	79.15
5	0-30-I-F-14-15-8-9-0	91.25	68.65
Total cost	Not feasible solution		

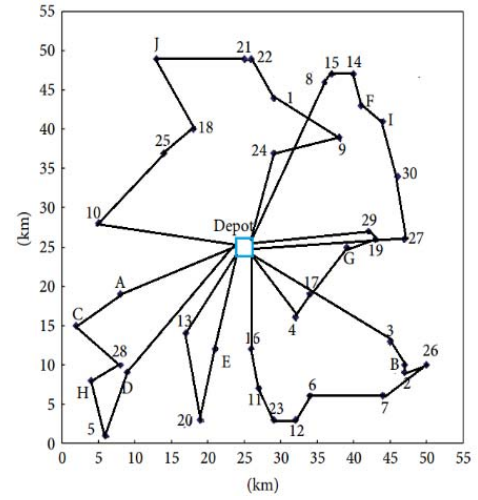


Fig. 7. Solution for the period 2

TABLE IV
THE GPU SOLUTION FOR THE PERIOD 1.

Vehicle	Route	Utilization %	Cost/Km
1	0-11-23-12-6-7-2-26-3-19-29-0	97.5	84.10
2	0-27-30-I-F-14-15-8-0	100	73.21
3	0-10-25-18-21-22-1-9-24-0	63.75	88.34
4	0-A-28-5-D-20-0	98.75	79.25
5	0-13-E-16-4-17-G-0	76.25	55.70
Total cost	380.62		

TABLE V
THE IVNS SOLUTION [5] FOR THE PERIOD 2.

Vehicle	Route	Utilization %	Cost/Km
1	0-17-7-3-B-2-26-27-19-29-G-0	> 100	81.35
2	0-4-6-12-23-11-16-0	70	50.81
3	0-24-1-22-21-18-J-25-10-0	96.25	93.17
4	0-13-E-20-5-28-D-H-C-A-0	> 100	88.69
5	0-30-9-I-F-14-15-8-0	91.25	74.45
Total cost	Not feasible solution		

TABLE VI
THE GPU SOLUTION FOR THE PERIOD 2.

Vehicle	Route	Utilization %	Cost/Km
1	0-3-B-2-26-7-6-12-23-11-16-0	96.25	77.51
2	0-D-5-H-28-C-A-0	100	75.97
3	0-27-30-I-F-14-15-8-0	100	73.21
4	0-10-25-18-J-21-22-1-9-24-0	98.75	99.24
5	0-13-20-E-0	42.5	47.60
6	0-29-19-G-17-4-0	75	47.60
Total cost	419.02		

B. Medium benchmarks

These medium benchmarks was compiled by Solomon [19]. The sample is divided into six categories: R1, R2, C1, C2, RC1, and RC2, each one contains 100 nodes. We report in Tab. 7 the results for the DVRP without time windows for the five data sets with 90%, 70%, 50%, 30%, and 10% dynamic degree. The dynamic degree is defined as :

$$Dod = \frac{nr}{n + nr} \times 100\%$$

The execution time of each instance is less than 10 seconds. Among the 30 instances, we get 21 new best results (bold entries in Table 7). For the other 8 we obtain solutions of poorer quality than (but close to) those of Xu and Lackner [5]. Our implementation made an improvement of 6% over Xu results and of 12% over Lackner results.

TABLE VII

GPU RESULTS VS. IVNS AND LACKNER RESULTS ON MEDIUM INSTANCES

No.	Dod (%)	IVNS [5]	Lackner [5]	GPU
R1	90	1343.82	1278.33	1332.39
	70	1331.35	1336.10	1259.96
	50	1293.81	1329.98	1200.92
	30	1286.63	1337.86	1006.53
	10	1259.18	1278.06	968.001
C1	90	1235.47	1479.60	1346.15
	70	1031.78	1261.30	1180.76
	50	1072.32	1236.06	1072.19
	30	970.80	1066.89	947.27
	10	895.68	996.35	830.15
RC1	90	1506.43	1475.21	1506.29
	70	1513.23	1488.44	1352.95
	50	1519.12	1448.07	1293.70
	30	1489.96	1439.71	1253.40
	10	1431.79	1426.89	1123.75
R2	90	1045.89	1193.33	1129.94
	70	1034.22	1116.93	1112.84
	50	1012.11	1138.78	957.15
	30	987.42	1085.42	943.96
	10	951.00	1052.85	803.48
C2	90	695.50	792.46	693.50
	70	671.86	743.78	675.42
	50	610.98	689.25	661.19
	30	655.23	632.33	649.12
	10	582.32	629.08	643.52
RC2	90	1351.21	1476.76	1312.85
	70	1256.32	1346.76	1190.74
	50	1189.13	1269.29	1151.82
	30	1151.35	1244.85	1072.65
	10	1183.11	1220.90	711.54
Total		33559	35511	31384

In Table 8 we compare the GPU implementation with the Ant Colony System (DVRP-ACS) proposed by Montemanni [2] and with the Genetic Algorithm (DVRP-GA) proposed by Hanshar [15]. Three separate VRP sources, namely tail* (for Taillard [20]), c* (for Christophides and Beasley [21]) and f* (for Fisher et al. [22]) are considered (the * is replaced by the size of the instance). The results show that the GPU implementation founds 17 new best solutions (out of 21) compared to the ACS and DVRP-GA. Our implementation made an improvement of 13% over Montemanni results [2] and 10% over Hanshar results [15].

C. Large benchmarks

Recently Uchoa Eduardo et al. [16] proposed a set of 100 more realistic instances with sizes from 100 to 1000. These instances vary according to the size and to four parameters : the depot position, the customer positions, the demands, and the average route length. As stated in [16] these benchmarks allow to better assess the performance of solution approaches in relation to different instance features, makes it possible to better discriminate competing algorithms. The distances are

TABLE VIII
GPU RESULTS VS. DVRP-ACS AND DVRP-GA RESULTS

Instance	DVRP-ACS [2]	DVRP-GA [15]	GPU
c100	973.26	961.10	918.81
c100b	944.23	881.92	830.15
c120	1416.45	1303.59	1068.14
c150	1345.73	1348.88	1209.20
c199	1771.04	1654.51	1456.67
c50	631.30	570.89	580.67
c75	1009.36	981.57	900.26
f134	15135.51	15528.81	12220.69
f71	311.18	301.79	256.11
tail100a	2375.92	2232.71	2184.83
tail100b	2283.97	2147.70	2033.33
tail100c	1562.30	1541.28	1394.32
tail100d	2008.13	1834.60	1707.11
tail150a	3644.78	3328.85	3519.65
tail150b	3166.88	2933.40	2888.06
tail150c	2811.48	2612.68	2457.22
tail150d	3058.87	2950.61	2960.36
tail75a	1843.08	1782.91	1645.49
tail75b	1535.43	1464.56	1356.63
tail75c	1574.98	1440.54	1313.54
tail75b	1472.35	1399.83	1356.63
Total	50876.23	49202.82	44257.87

two-dimensional Euclidean. Depot and costumers have integer coordinates corresponding to points in a $[0, 1000] \times [0, 1000]$. Each instance is characterized by the following attributes: number of customers, depot positioning, customer positioning, demand distribution, and average route size. Three different positions for the depot are considered : Central (C), Eccentric (E), Random (R). Three alternatives for customer positioning are considered : Random (R), Clustered (C) and Random-clustered (RC). Seven options of demand distributions are considered (see [16] for more detail on these benchmarks).

The authors report the results of the iterated local search based metaheuristic algorithm of Subramanian et al. [23] and the unified hybrid genetic search of Vidal et al. [24] [25] on these instances for the static VRP. In Tables 9 and 10 we report the results of our approach on these instances where we assume that, for each instance, the first 50% customers are static (period 0) and the remaining are dynamic (period 1). Note that the results of [16] concern the static VRP and should be better than ours, despite this we have improved 8 (bold entries).

Finally, we generated our own instances with 1000 to 3000 nodes as follows. Integer coordinates of the customers are randomly generated in the square $[0, 1000] \times [0, 1000]$. Vehicle capacity is equal to $\lfloor n/8 \rfloor \times 200$. Each request is a random number of $[1, 200]$. The location of the distribution center is (500, 500). For each instance, 50% static requests and 50% dynamic requests randomly generated.

All new requests, of an instance problem, are served in period 1. We report in Table 4 the costs of the GPU solutions and the corresponding execution times which increases quasi-linearly with n . Since, our modest GPU has two SMs these times will be divided by $\frac{P}{2}$ on a GPU with P SMs (for instance by 42 on volta100).

TABLE IX
GPU RESULTS VS. UCHOA RESULTS [16] FOR LARGE INSTANCES

#	N	Dep	Cust	Dem	Cost	Time
1	100	R	RC(7)	1-100	28988.52	0.16
2	105	E	C(3)	50-100	27259.48	0.17
3	109	C	R	5-10	16634.21	0.21
4	114	C	R	SL	13819.57	0.25
5	119	E	RC(8)	U	14209.14	0.25
6	124	R	C(5)	Q	58473.38	0.31
7	128	E	RC(8)	1-10	29991.24	0.36
8	133	R	C(4)	Q	11513.84	0.40
9	138	C	R	5-10	16835.94	0.52
10	142	E	R	1-100	18252.95	0.54
11	147	R	RC(7)	1-10	32946.36	0.64
12	152	C	C(3)	SL	22638.54	0.70
13	156	R	C(3)	U	17057.27	0.74
14	161	C	RC(8)	50-100	15435.26	0.81
15	166	E	R	5-10	22699.27	0.87
16	171	C	RC(5)	Q	48231.55	0.92
17	175	E	R	SL	50055.37	0.98
18	180	R	C(6)	U	26349.20	1.03
19	185	R	R	50-100	26673.68	1.08
20	189	E	C(3)	1-10	17136.42	1.19
21	194	C	RC(5)	1-100	45700.50	1.25
22	199	R	C(8)	Q	58966.19	1.29
23	203	C	RC(6)	50-100	21598.04	1.33
24	208	E	R	5-10	33792.54	1.40
25	213	C	C(4)	1-100	12200.10	1.46
26	218	E	R	U	100191.75	1.51
27	222	R	RC(5)	1-10	41043.86	1.54
28	227	R	C(8)	SL	27098.66	1.59
29	232	C	RC(7)	Q	22163.71	1.62
30	236	E	R	U	29846.72	1.67
31	241	E	R	1-10	83537.25	1.73
32	246	C	C(4)	SL	32202.90	1.79
33	250	R	RC(3)	5-10	40131.30	1.85
34	255	C	C(8)	50-100	25376.91	1.91
35	260	E	R	1-100	31142.23	1.98
36	265	R	RC(6)	5-10	73998.98	2.01
37	269	C	RC(5)	50-100	38878.11	2.09
38	274	R	C(3)	U	22785.09	2.12
39	279	E	R	SL	36621.25	2.16
40	283	R	C(8)	01-10	22287.42	2.24
41	288	E	RC(7)	Q	97724.07	2.29
42	293	C	R	1-100	52220.10	2.36
43	297	R	R	1-10	35802.08	2.42
44	302	C	C(8)	1-100	28144.35	2.49
45	307	E	RC(6)	SL	30139.12	2.53
46	312	R	RC(3)	Q	97460.29	2.69
47	316	E	C(4)	U	76568.83	2.83
48	321	C	R	50-100	36472.85	2.92
49	326	R	RC(7)	5-10	32312.96	3.04
50	330	E	R	U	35457.97	3.19

TABLE X
GPU RESULTS VS. UCHOA RESULTS (CONTINUED)

#	N	Dep	Cust	Dem	Cost	Time
51	335	E	R	Q	140138.64	3.31
52	343	C	RC(7)	5-10	48020.46	3.61
53	350	C	C(4)	1-100	30990.57	3.74
54	358	E	RC(7)	1-10	54416.97	3.81
55	366	R	C(4)	SL	29801.32	4.21
56	375	E	R	U	148816.35	4.56
57	383	R	R	50-100	71583.70	4.98
58	392	C	RC(5)	5-10	44617.00	5.32
59	400	E	C(6)	Q	67728.76	5.54
60	410	R	C(5)	SL	25289.44	5.74
61	419	C	RC(3)	1-10	112606.52	5.93
62	428	R	R	50-100	73386.80	6.14
63	438	C	RC(8)	U	47018.64	6.38
64	448	E	R	1-100	61665.39	6.52
65	458	C	C(4)	Q	30520.36	6.75
66	468	E	R	50-100	235577.01	6.98
67	479	R	C(8)	5-10	95416.05	7.57
68	490	R	RC(6)	1-100	71902.34	8.03
69	501	E	C(6)	U	66636.25	9.57
70	512	C	RC(4)	1-10	31501.54	10.09
71	523	R	R	SL	164327.53	10.57
72	535	C	C(7)	Q	102869.57	11.67
73	547	E	R	U	91417.21	12.78
74	560	C	RC(7)	1-10	52759.54	14.40
75	572	E	C(3)	SL	49818.60	16.98
76	585	R	RC(4)	5-10	198978.34	19.54
77	598	R	R	50-100	115916.26	21.76
78	612	C	R	1-100	68446.44	24.94
79	626	E	C(5)	5-10	64198.83	26.12
80	640	E	RC(8)	50-100	68939.58	28.91
81	654	C	C(4)	U	107834.96	30.84
82	669	R	R	SL	159412.21	33.18
83	684	C	RC(6)	Q	79876.54	37.46
84	700	E	RC(7)	1-10	88109.75	43.74
85	715	R	C(3)	1-100	46205.78	45.80
86	732	C	R	1-10	143343.25	50.06
87	748	R	C(8)	1-100	84294.70	54.78
88	765	E	RC(7)	SL	117647.95	59.23
89	782	R	R	Q	82568.79	65.93
90	800	E	R	U	81123.57	73.81
91	818	C	C(6)	50-100	172215.51	77.10
92	836	R	RC(7)	5-10	201273.73	84.77
93	855	C	RC(3)	U	102658.58	90.23
94	875	E	C(5)	1-100	101081.64	96.67
95	894	R	R	50-100	70399.38	104.78
96	915	E	RC(6)	5-10	338097.96	116.23
97	935	C	R	SL	155051.89	127.55
98	956	R	RC(4)	U	101118.41	140.36
99	978	E	C(6)	Q	123092.28	159.78
100	1000	R	R	1-10	82142.17	176.42

VII. CONCLUSION

In this paper, we presented a GA based approach for Large Dynamic VRP. The method improves iteratively an initial solution via genetic operators (crossover, re-schedule, mutation) that we introduced. We implemented this method on the GPU in order to solve large instances of this problem. We compared our GPU implementation to others recent published approaches : GA based approach, Ant Colony System, etc. The experimental results showed that our implementation was able to find high quality solutions (when compared to the other approaches) and to solve large instances up to 3000 nodes. To our knowledge, this is the first attempt to solve large DVRP on the GPU using evolutionary algorithm and seems to

be efficient and clearly outperforms the CPU implementation according to the experimental results.

We actually work to extend this approach (method and GPU implementation) to other variants and class of DVRP including time-window, uncertainty and green concepts. Also, we are thinking about improving the CUDA implementation by limiting the use of the global memory of the GPU. Another future work is the use of a cluster of GPUs with MPI (Message Passing Interface).

ACKNOWLEDGMENTS

We thank Dr Montemanni R. for instances data set provided. This work is partially supported by the Classe2 project.

TABLE XI
GPU RESULTS FOR LARGE INSTANCES

Instance size	Cost solution	Time execution (s)
1000	70626.21	187.28
1100	68488.65	108.10
1200	79750.60	174.45
1300	81232.98	242.53
1400	83799.35	343.84
1500	88531.78	448.99
1600	88123.15	501.11
1700	95355.32	609.23
1800	107430.60	747.98
1900	111741.55	976.61
2000	119407.52	1181.13
2100	129343.18	1249.37
2200	141307.29	1270.61
2300	137464.59	1356.31
2400	144077.32	1493.69
2500	157625.26	1568.66
2600	147283.43	1674.90
2700	147292.35	1757.28
2800	170980.03	1885.54
2900	161612.31	2028.76
3000	189496.37	2132.92

REFERENCES

- [1] K. Meesuptaweekoon and P. Chaovalitwongse, "Dynamic vehicle routing problem with multiple depots," *Engineering Journal*, vol. 18(4), pp. 135–149, 2014.
- [2] R. Montemanni, L. M. Gambardella, A. E. Rizzoli, and A. V. Donati, "A new algorithm for a dynamic vehicle routing problem based on ant colony system," *J. Comb Optim*, vol. 10, p. 327343, 2005.
- [3] R. M. Branchini, V. A. Armentano, and A. Lkktangen, "Adaptive granular local search heuristic for a dynamic vehicle routing problem," *Computers and Operations Research*, vol. 36(11), p. 2955–2968, 2009.
- [4] V. Pillac, M. Gendreau, C. Guret, and A. L. Medaglia, "A review of dynamic vehicle routing problems," *European Journal of Operational Research*, vol. 225 (1), pp. 1–11, 2013.
- [5] Y. Xu, L. Wang, and Y. Yang, "Dynamic vehicle routing using an improved variable neighborhood search algorithm," *Journal of Applied Mathematics*, 2013.
- [6] M. Gendreau and J. Y. Potvin, "Issues in real-time fleet management," *Transportation Science*, vol. 38.4, 2004.
- [7] L. Abdatecola, M. P. Fantì, and W. Ukovich, "A review of new approach for dynamic vehicle routing problem," in *IEEE CASE*, pp. 361–366, 2016.
- [8] F. Campeotto, A. Dovier, F. Fioretto, and E. Pontelli, "A gpu implementation of large neighborhood search for solving constraint optimization problems," *Proceedings of the Twenty-first European Conference on Artificial Intelligence*, pp. 189–194, 2014.
- [9] A. Arbelaez and P. Codognet, "A gpu implementation of parallel constraint-based local search," in *PDP2014, 22nd Euromicro International Conference*, p. 648–655, 2014, February.
- [10] P. Uthayopas, G. Srimool, J. Pichitlamken, and N. Benjamas, "Speeding up the pickup and delivery problem with time windows using gpu cluster," *International Journal of Engineering and Industries*, vol. 4(2), 2013.
- [11] A. Benaini and A. Berrajaa, "Solving the dynamic vehicle routing problem on gpu," in *IEEE xplore GOL'16*, pp. 1–6, 2016.
- [12] T. V. Luong, N. Melab, and E. Talbi, "Gpu computing for parallel local search metaheuristic algorithms," *IEEE Transactions on Computers*, vol. 62(1), pp. 173–185, 2012.
- [13] E. G. Talbi and G. Hasle, "Metaheuristics on gpus," *J. Parallel Distrib. Comput.*, vol. 73(1), 2013.
- [14] F. J. Diego, E. M. Gmez, M. Ortega-Mier, and . Garca-Snchez, "Parallel cuda architecture for solving de vrp with aco," *Industrial Engineering, Innovative Networks*, pp. 385–393, 2012.
- [15] F. T. Hanshar and B. M. Ombuki-Berman, "Dynamic vehicle routing using genetic algorithms," *Applied Intelligence*, vol. 27(1), pp. 89–99, 2007.
- [16] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, and A. Subramanian, "New benchmark instances for the capacitated vehicle routing problem," *European Journal of Operational Research*, vol. 257(3), pp. 845–858, 2017.
- [17] M. Naeem and B. Ombuki-Berman, "An efficient genetic algorithm for the uncapacitated single allocation hub location problem," in *Evolutionary Computation (CEC), 2010 IEEE Congress on IEEE*, pp. 1–8, 2010.
- [18] N. Matloff, "Programming on parallel machines, university of california, <http://heather.cs.ucdavis.edu/matloff/parprocbook.pdf>,"
- [19] M. Solomon, "Algorithms for the vehicle routing and scheduling problems with time window constraints," *Operations research*, vol. 35(2), pp. 254–265, 1993.
- [20] E. Taillard, "Parallel iterative search methods for vehicle routing problems," *Networks*, vol. 23(8), pp. 661–673, 1993.
- [21] N. Christofides and J. E. Beasley, "The period routing problem," *Networks*, vol. 14(2), pp. 237–256, 1984.
- [22] M. Fisher, "Vehicle routing," *Handbooks in operations research and management science*, vol. 8, pp. 1–33, 1995.
- [23] A. Subramanian, E. Uchoa, and L. S. Ochi, "A hybrid algorithm for a class of vehicle routing problems," *European Journal of Operational Research*, vol. 40(10), pp. 2519–2531, 2013.
- [24] T. Vidal, T. G. Crainic, M. Gendreau, N. Lahrichi, and W. Rei, "A hybrid genetic algorithm for multidepot and periodic vehicle routing problems," *Operations Research*, vol. 60(3), pp. 611–624, 2012.
- [25] T. Vidal, T. G. Crainic, M. Gendreau, and C. Prins, "A unified solution framework for multi-attribute vehicle routing problems," *European Journal of Operational Research*, vol. 234(3), p. 658673, 2014.